Primer: Temporal-Difference CCP (TD-CCP)

Adusumilli and Eckardt (2025)

econirl package documentation

1 Context and Problem

CCP estimation (Hotz and Miller 1993) avoids the inner Bellman loop of NFXP by inverting observed choice probabilities into value differences. The inversion requires computing the expected future value $\mathbb{E}[V(s') \mid s, a]$, which in turn requires the transition density $p(s' \mid s, a)$. When the state space is continuous or high-dimensional, estimating this density is the binding constraint. Kernel methods suffer from the curse of dimensionality, and discretization introduces approximation bias that grows with the number of state variables.

Adusumilli and Eckardt (2025) eliminate the transition density entirely. Their key insight is that the CCP value function components $h(a, x)$ and $g(a, x)$ satisfy temporal-difference fixed-point equations that depend only on sample expectations over observed transitions (x_t, a_t, x_{t+1}), not on the density $p(x' \mid x, a)$. This is the same idea that makes TD learning in reinforcement learning model-free: replace $\mathbb{E}_p[V(s')]$ with the sample next state $V(s'_{t+1})$.

The estimator comes in two flavors. The linear semi-gradient method approximates h and g with polynomial basis functions and solves a single linear system, running in sub-second time. The neural approximate value iteration (AVI) method uses neural networks for h and g, iterating Bellman-like regression targets until convergence. Both produce locally robust standard errors at the parametric $\sqrt{n}$ rate, even though the first-stage h and g estimates converge at slower nonparametric rates.

TD-CCP is the only estimator in econirl that requires neither transition densities nor an inner Bellman loop. NFXP and CCP require known transitions. NNES and SEES require transitions for the Bellman target. AIRL avoids transitions but does not produce valid standard errors. TD-CCP provides all three: no transitions, scalability, and valid inference.

The name "temporal-difference CCP" reflects the connection to TD learning in reinforcement learning. Just as TD(0) updates value estimates using the observed next state rather than the full expectation over the transition kernel, TD-CCP updates value function components h_k and g using observed next transitions (x_{t+1}, a_{t+1}) rather than the density $p(x' \mid x, a)$. The semi-gradient fixed-point equation (3) is the structural econometrics analog of the Tsitsiklis and Van Roy (1997) linear TD method with function approximation.

2 Model and Notation

Symbol	Meaning
$x \in \mathcal{X}$	State (continuous or discrete)
$a \in \mathcal{A}$	Action (discrete, $\lvert\mathcal{A}\rvert$ choices)
$\beta \in [0,1)$, $\sigma > 0$	Discount factor, logit scale
$\phi_k(a,x) \in \mathbb{R}$	Feature k for action a at state x
$\theta \in \mathbb{R}^K$	Structural parameters
$P(a \mid x)$	Conditional choice probability (CCP)
$e(a,x) = \gamma_E - \ln P(a \mid x)$	Emax correction (Euler constant $\gamma_E \approx 0.577$)
$h_k(a,x)$	Expected discounted feature k under action a
$g(a,x)$	Expected discounted emax under action a
$z_k(a,x), z_g(a,x)$	Pseudo-outcome for h_k, g
$\Psi(a,x) \in \mathbb{R}^{K_\Psi}$	Basis functions for semi-gradient
$\omega_k \in \mathbb{R}^{K_\Psi}$	Basis coefficients for h_k
$\lambda_k(a,x)$	Backward value function (for robust SEs)

The CCP decomposition of Hotz and Miller (1993) writes the choice-specific value as

$$Q(a,x;\theta) \;=\; \sum_{k=1}^{K} \theta_k \, h_k(a,x) \;+\; \sigma \, g(a,x), \tag{1}$$

where $h_k(a,x) = \mathbb{E}\big[\sum_{t=0}^{\infty} \beta^t \phi_k(a_t,x_t) \mid a_0 = a, x_0 = x\big]$ is the expected discounted feature k and $g(a,x) = \mathbb{E}\big[\sum_{t=1}^{\infty} \beta^t e(a_t,x_t) \mid a_0 = a, x_0 = x\big]$ is the expected discounted emax. The pseudo-log-likelihood is

$$Q_n(\theta) \;=\; \frac{1}{n}\sum_{i=1}^{n} \ln \pi\big(a_i \mid x_i;\, \theta,\, h,\, g\big), \qquad \pi(a \mid x) = \frac{\exp\big(Q(a,x;\theta)/\sigma\big)}{\sum_{a'} \exp\big(Q(a',x;\theta)/\sigma\big)}. \tag{2}$$

The key objects h_k and g satisfy temporal-difference fixed-point equations that do not involve the transition density.

3 Identification and Theory

Proposition 1 (TD fixed point for h)**.** *The component $h_k(a,x)$ satisfies the fixed-point equation $h_k(a,x) = z_k(a,x) + \beta\,\mathbb{E}[h_k(a',x') \mid a,x]$, where $z_k(a,x) = \phi_k(a,x) + \beta \sum_{a'} P(a' \mid x)\big[\phi_k(a',x) - \phi_k(a,x)\big]$ is the pseudo-outcome and (a',x') is the next observed state-action pair. The expectation is over the data-generating process, not over a model transition density.*

The semi-gradient method approximates $h_k(a,x) \approx \Psi(a,x)^\top \omega_k$ and solves

$$\hat{\omega}_k \;=\; \Big[\frac{1}{n}\sum_{i=1}^{n} \Psi_i\big(\Psi_i - \beta\,\Psi_i'\big)^\top\Big]^{-1} \frac{1}{n}\sum_{i=1}^{n} \Psi_i \, z_{k,i}, \tag{3}$$

where $\Psi_i = \Psi(a_i,x_i)$ and $\Psi_i' = \Psi(a_i',x_i')$ are basis evaluations at current and next transitions. This is a single matrix inversion, shared across all K components.

Proposition 2 (Locally robust inference)**.** *Define the backward value function $\lambda_k(a,x)$ as the solution to the adjoint equation $(I - \beta F_\pi^\top)\lambda_k = \partial Q/\partial h_k$. The locally robust moment condition is*

$$\zeta_i(\theta) \;=\; m_i(\theta) \;+\; \sum_k \theta_k \, \lambda_k(a_i',x_i') \, \delta_{h,k,i} \;+\; \sigma \, \lambda_g(a_i',x_i') \, \delta_{g,i}, \tag{4}$$

where m_i is the standard score, $\delta_{h,k,i} = z_{k,i} + \beta\,\hat{h}_k(a_i',x_i') - \hat{h}_k(a_i,x_i)$ is the TD error for h_k, and $\delta_{g,i}$ is defined analogously for g. Under regularity conditions, $\hat{\theta}$ is asymptotically normal at rate $\sqrt{n}$ with sandwich variance $V = (G^\top \Omega^{-1} G)^{-1}$, even when $\hat{h}$ and $\hat{g}$ converge at slower nonparametric rates.

The key property is Neyman orthogonality: the moment condition ζ_i has zero derivative with respect to the nuisance functions h and g at the true values. This means first-order errors in $\hat{h}$ and $\hat{g}$ drop out of the influence function for

$\hat{\theta}$, so the structural parameters inherit the parametric $\sqrt{n}$ rate even when the nonparametric first stage converges at a slower rate such as $n^{-1/3}$ or $n^{-2/5}$. Without the correction terms in (4), naive standard errors would understate uncertainty by ignoring the estimation error in h and g.

Cross-fitting (splitting data into two folds, estimating h, g on one fold and θ on the other) eliminates the Donsker condition that would otherwise restrict the complexity of the first-stage estimator. On tabular problems where the state space is finite, the standard approach without cross-fitting also produces valid SEs because the CCP estimates converge at the parametric rate. The paper recommends cross-fitting for continuous-state applications where nonparametric rates apply.

The identification conditions for θ are the same as for NFXP: the feature matrix must have full column rank, and the discount factor β must be fixed a priori. The TD-CCP approach does not weaken or strengthen the identification requirements relative to standard CCP. What it changes is the computational requirements: the transition density is no longer needed for estimation, only for counterfactual simulation after estimation.

4 Estimation Algorithm

Algorithm 1: TD-CCP with Semi-Gradient (Adusumilli and Eckardt 2025)

Input: Panel $\{(x_i, a_i, x_i', a_i')\}_{i=1}^n$, features ϕ, basis Ψ, discount β

Cross-fitting: split data into folds $\mathcal{I}_1, \mathcal{I}_2$ `// _estimate_with_cross_fitting()`

for *fold* $\ell = 1, 2$ **do**

  Step 1: Estimate CCPs `// _estimate_ccps()`

  $\hat{P}(a \mid x) \leftarrow$ frequency counts (or logit on polynomial features)

  $e_i \leftarrow \gamma_E - \ln \hat{P}(a_i \mid x_i)$ for each observation

  Step 2: Build pseudo-outcomes

  $z_{k,i} \leftarrow \phi_k(a_i, x_i) + \beta \sum_{a'} \hat{P}(a' \mid x_i)[\phi_k(a', x_i) - \phi_k(a_i, x_i)]$

  $z_{g,i} \leftarrow e_i + \beta \sum_{a'} \hat{P}(a' \mid x_i)[e(a', x_i) - e(a_i, x_i)]$

  Step 3: Solve semi-gradient `// _semigradient_solve()`

  $A \leftarrow \frac{1}{|\mathcal{I}_\ell|} \sum_{i \in \mathcal{I}_\ell} \Psi_i (\Psi_i - \beta \Psi_i')^\top$

  for $k = 1, \ldots, K$ *and for* g **do**

    $\hat{\omega}_k \leftarrow A^{-1} \big[\frac{1}{|\mathcal{I}_\ell|} \sum_{i \in \mathcal{I}_\ell} \Psi_i \, z_{k,i}\big]$ `// Eq. 3`

  end

  Step 4: Partial MLE on other fold `// _partial_mle()`

  $\hat{\theta}^{(\ell)} \leftarrow \arg\max_\theta \frac{1}{|\mathcal{I}_{-\ell}|} \sum_{i \in \mathcal{I}_{-\ell}} \ln \pi(a_i \mid x_i;\, \theta,\, \hat{h},\, \hat{g})$

  Step 5: Backward value function `// _compute_backward_value()`

  Solve $(I - \beta \hat{F}_\pi^\top) \hat{\lambda}_k = \partial Q / \partial h_k$ for each k

  Step 6: Robust SEs `// _compute_robust_se()`

  Compute $\hat{\zeta}_i$ via (4)

  $\hat{V}^{(\ell)} \leftarrow (\hat{G}^\top \hat{\Omega}^{-1} \hat{G})^{-1}$

end

$\hat{\theta} \leftarrow \frac{1}{2}(\hat{\theta}^{(1)} + \hat{\theta}^{(2)}), \quad \hat{V} \leftarrow \frac{1}{2}(\hat{V}^{(1)} + \hat{V}^{(2)})$

Output: $\hat{\theta}$, $\hat{V}$ for robust SEs, $\hat{h}$, $\hat{g}$ for diagnostics

Code annotations reference methods in `src/econirl/estimation/td_ccp.py`. The matrix A in Step 3 is shared across all components, so the cost is one matrix inversion plus $K + 1$ back-solves. Default configuration: $K_\Psi = 8$ polynomial basis, 2-fold cross-fitting, robust SEs enabled. For neural AVI, replace Step 3 with iterative regression networks (`_neural_avi_solve()`).

5 Simulation

Table 1: Firm value model (100 states, $\beta = 0.95$, 15,000 obs). True: $\theta_v = 5.0$, $\theta_{FC} = -1.0$. CCP must discretize the continuous state; TD-CCP does not.

Method	$\hat{\theta}_v$	$\hat{\theta}_{FC}$	RMSE	Acc	Time (s)
True values	5.0	-1.0	—	—	—
BC	—	—	—	97.0%	—
CCP ($K = 5$ bins)	4.6670	-0.9631	0.2369	99.0%	5.5
CCP ($K = 100$ bins)	4.2893	-0.7248	0.5389	97.0%	4.3
TD-CCP	4.9653 (0.0540)	-0.9301 (0.0233)	0.0552	99.0%	2.2

The experiment simulates a firm value model where productivity $z \in [0, 1]$ follows an AR(1) process with persistence $\rho = 0.8$. Firms choose to operate ($a = 1$) or exit ($a = 0$) each period. Flow utility is $u(1, z; \theta) = \theta_v z + \theta_{FC}$ and $u(0, z; \theta) = 0$, where $\theta_v = 5.0$ and $\theta_{FC} = -1.0$. The state is discretized to 100 bins approximating the continuous process. The dataset has 15,000 observations from 3,000 individuals over 5 periods with $\beta = 0.95$.

CCP must discretize the state into K coarse bins and estimate the transition matrix $\hat{P}(z' \mid z, a)$ by frequency counting. With $K = 5$ bins, each bin spans a productivity range of 0.2, averaging over a utility variation of $\theta_v \times 0.2 = 1.0$, which is larger than both structural parameters combined. The coarse features bias the parameter estimates away from the truth. The natural fix is to increase K: at $K = 100$ bins (the same resolution as the fine grid, so no coarsening at all), the AR(1) noise spreads each source state across roughly 17 destination bins. With 15,000 observations and two actions, each source state receives about 75 observations; spread across 17 active transition cells, that yields roughly 4 observations per cell. The frequency estimate $\hat{P}$ is nearly noise, and the NPL propagates this noise directly into the parameter estimates. There is no safe choice of K: coarser bins introduce feature bias, finer bins introduce transition estimation variance. This is the Goldilocks trap for discretized CCP. TD-CCP uses degree-8 polynomial basis functions over $z_{\text{norm}} \in [0, 1]$ and never forms $\hat{P}$ at all.

Table 1 shows the outcome. TD-CCP achieves RMSE of 0.0552, with robust standard errors in parentheses. CCP with $K = 5$ bins reaches RMSE of 0.2369 (feature bias dominates), 4.3 times worse. CCP with $K = 100$ bins reaches RMSE of 0.5389 (transition noise dominates), 9.8 times worse. Finer discretization makes things worse, not better, once the transition matrix is too sparse to estimate reliably. Behavioral cloning reaches 97.0% policy accuracy but recovers no structural parameters.

```
from econirl.estimation.td_ccp import TDCCPEstimator, TDCCPConfig

config = TDCCPConfig(method="semigradient", basis_dim=8,
    ccp_method="logit", robust_se=True)
result = TDCCPEstimator(config=config).estimate(
    panel, utility, problem, transitions)
```

Code. `tdccp_run.py` generates Table 1. Estimator: `src/econirl/estimation/td_ccp.py`.

```
.venv/bin/python papers/econirl_package/primers/tdccp/tdccp_run.py
```